



GUIA DE DISEÑO DE APIS

**LINEAMIENTOS Y CONVENCIONES PARA LA DEFINICIÓN Y
GOBIERNO DE APIS**

**ARQUITECTURA DE INTEGRACIÓN Y SERVICIOS
VERSIÓN 1.6.1**

Contenido

1.	Control de versiones	3
2.	Control de aprobaciones.....	3
3.	Introducción	4
3.1.	Propósito.....	4
3.2.	Objetivo.....	4
3.3.	Alcance	4
3.4.	Audiencia	6
3.5.	Relación con otros documentos	6
4.	Principios de diseño	6
5.	Especificaciones técnicas	6
5.1.	Protocolos y estándares.....	7
5.2.	Nomenclatura y convenciones.....	8
5.3.	Modelado de recursos	13
5.4.	Operaciones	14
6.	Estructura de mensaje.	15
7.	Manejo de errores	19
7.1.	Códigos de respuesta HTTP.....	19
7.2.	Estructura de cuerpo del mensaje de error	20
8.	Seguridad	21
8.1.	Autenticación y autorización	21
8.2.	Gestión segura de tokens	22
8.3.	Encriptación en tránsito y en reposo	23
8.4.	Exposición de Datos	23
8.5.	Protección contra abuso	23
8.6.	Otros controles recomendados	23
9.	Versionado y ciclo de vida.....	23
9.1.	Estrategia de versionado	23
9.2.	Semántica de versiones	23
9.3.	Clasificación de Cambios.....	24
9.4.	Estrategia de Versionado	24
9.5.	Comunicación de Versiones MINOR y PATCH.....	24
9.6.	Garantía de compatibilidad	25
9.7.	Deprecación y retiro	25

10.	Calidad y pruebas.....	25
10.1.	Pruebas	25
10.2.	Pruebas de carga y resiliencia	25
11.	Observabilidad	25
11.1.	Logs estructurados.....	25
11.2.	Trazabilidad distribuida.....	25
11.3.	Métricas	26
12.	Gobierno y cumplimiento	26
12.1.	Revisión por Chapter.....	26
12.2.	Checklist de cumplimiento.....	26
12.3.	Roles y responsabilidades	26
13.	Ejemplos y antipatronos	26
13.1.	Buenas prácticas (ejemplos)	26
13.2.	Antipatronos a evitar	26
14.	Glosario y referencias	27
14.1.	Definiciones clave	27
14.2.	Referencias y estándares	27

1. Control de versiones

Descripción	Versión	Autor	Fecha
Release interno en Arquitectura	1.0	Otto Alvarado	09/10/2025
Observaciones de equipos de Gobierno y delivery y Arquitectura de software	1.1	Otto Alvarado	17/10/2025
Observaciones de equipos de Gobierno y delivery y Arquitectura de software	1.2	Otto Alvarado	23/10/2025
Observaciones de formato. Se agrego prefijo dirección a URL.	1.3	Otto Alvarado	29/10/2025
Se agrego sufijo path integración terceros.	1.4	Otto Alvarado	10/11/2025
Observaciones Equipo Arquitectura Ejemplo valores headers. Esquema de seguridad.	1.5	Otto Alvarado	01/12/2025
Agregar especificaciones para APIs de Experiencia	1.6	Otto Alvarado	09/02/2026
Agregar obligatoriedad, longitud min y max para campos body y código de error 415	1.6.1	Otto Alvarado	12/08/2026

2. Control de aprobaciones

Revisado por	Versión revisada	Aprobado por	Fecha
Arquitectura de integración y servicios	1.6	Óscar Vasquez	08/10/2025

3. Introducción

3.1. Propósito

Definir directrices y principios que orienten el diseño, desarrollo y gestión de las APIs corporativas, asegurando alineamiento con la arquitectura empresarial, la estrategia digital y las mejores prácticas del mercado.

3.2. Objetivo

Definir lineamientos y convenciones para diseño, construcción, documentación y operación de APIs en Interbank. Busca garantizar la consistencia, seguridad, escalabilidad y gobernanza en la exposición y consumo de servicios digitales, promoviendo la innovación y la transformación digital del banco sin comprometer la confianza, la estabilidad ni el cumplimiento normativo.

Asimismo, establece convenciones de nomenclatura, protocolos, estilos, versionado, estructura de mensajes, ciclo de vida, documentación, manejo de errores y seguridad que deberán cumplirse en todas las APIs corporativas.

3.3. Alcance

Aplica a todas las APIs desarrolladas y operadas en Interbank, incluyendo:

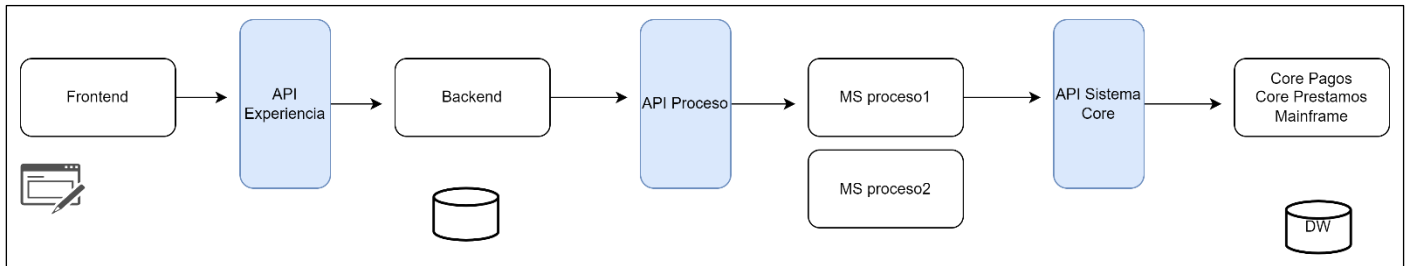
Clasificación en base a exposición:

Tipo	Descripción
APIs internas/privadas	APIs expuestas en nuestro tenant/cuentas de cloud o en onpremise para ser consumidas por aplicaciones internas backoffice, middle office y front office
APIs partners/terceros	APIs compartidas con socios o terceros autorizados
APIs públicas	APIs abiertas a desarrolladores externos o comunidades

Clasificación según capa funcional:

Según modelo de Mulesoft (API-Led Connectivity) las APIs se clasifican en:

Tipo	Descripción
APIs de experiencia	APIs expuestas en la capa de experiencia, que son consumidos por el front de los canales. también conocido como “System of engagement”. Los proveedores de estas APIs son los BFF y/o microservicios exclusivos de los canales.
APIs de aplicación	APIs que se usan dentro del dominio de una aplicación basado en microservicios, la comunicación interna se da entre APIs de aplicación.
APIs de proceso	APIs expuestas en la capa de proceso que son consumidos por los backends y/o microservicios de los canales. Son APIs reutilizables. Los proveedores de estas son otros microservicios o componentes que tienen lógica de negocio cross reutilizable.
APIs de sistema	APIs expuestas en la capa de experiencia que generalmente son consumidos por los backends y/o microservicios con funcionalidades cross. Los proveedores de estas APIs son sistemas core, legacy, mainframe, SaaS o repositorios finales.



Aplicación sobre APIs transversales

Se establece que las APIs de negocio, que comprenden las APIs de Proceso y las APIs de Sistema, son el principal foco de aplicación y cumplimiento de los lineamientos de diseño, definición y estructura detallados en este documento.

La rigurosidad y adherencia estricta a los estándares definidos son imperativas para estas APIs, dada su naturaleza de:

- **Core:** Encapsulan la lógica de negocio esencial y la conectividad a los sistemas de registro.
- **Reutilización:** Están diseñadas para ser consumidas por múltiples aplicaciones y experiencias a lo largo del ecosistema.
- **Estabilidad:** Su correcta definición asegura la integridad y consistencia del modelo de datos y las transacciones operativas.

Para asegurar la máxima reutilización, la consistencia de la arquitectura de datos y la estabilidad de la plataforma, el Equipo de Gobierno y Delivery con los Chapters de Integración ejercerá injerencia en la fase de Aprobación del Diseño y la fase de Revisión de Cumplimiento de estas APIs.

Aplicación sobre las APIs de Experiencia/Aplicación

Las APIs de Experiencia (y las APIs de Aplicación) recibirán una mayor flexibilidad en la aplicación de los lineamientos establecidos.

Esta consideración se fundamenta en las siguientes características intrínsecas de las APIs de Experiencia:

- **Dependencia Específica:** Su diseño está altamente orientado a satisfacer los requisitos y la funcionalidad específica de la solución de consumo (la aplicación final).
- **Particularidades:** Frecuentemente presentan particularidades y adaptaciones que optimizan la usabilidad o el rendimiento de una experiencia de usuario concreta.
- **Dominio de Aplicación:** Operan primariamente en el dominio de cada aplicación, actuando como fachadas o adaptadores que orquestan el consumo de las APIs de Negocio subyacentes.
- **Baja Reutilización:** Por su especialización, su nivel de reutilización a través de la organización suele ser limitado.

La responsabilidad integral del diseño, implementación y validación de las **APIs de Experiencia** y **APIs de Aplicación** recae **exclusivamente** en el **Equipo de Arquitectura de Software**. Para garantizar la máxima agilidad y autonomía, el Equipo de Gobierno de APIs **no ejerce injerencia** en las fases de diseño o validación de estas interfaces. Su única función de supervisión es asegurar el consumo conforme y seguro de las APIs de Negocio subyacentes.

Tipo de API	Prefijo
APIs de experiencia	exp
APIs de aplicación	app
APIs de proceso	proc
APIs de sistema	sys

3.4. Audiencia

La información vertida en este documento debe ser de interés para todos los siguientes equipos de la VPO de Interbank: Arquitectura, Crews, Servicios TI y Ciberseguridad.

3.5. Relación con otros documentos

Lineamientos de seguridad de APIs.

4. Principios de diseño

Principio	Descripción
API first	Toda iniciativa digital debe partir del diseño de APIs antes de la implementación del backend, garantizando contratos claros, documentados y reutilizables.
Seguridad como pilar fundamental	Las APIs deben diseñarse bajo el principio de “ security by design ”, asegurando autenticación, autorización, cifrado y cumplimiento normativo desde el inicio.
Reutilización y modularidad	Se debe priorizar la construcción de servicios reutilizables para evitar duplicidad de lógica y maximizar la eficiencia operativa. Se debe priorizar la construcción de servicios reutilizables para evitar duplicidad de lógica y maximizar la eficiencia operativa.
Desacoplamiento y escalabilidad	Las APIs deben diseñarse para minimizar dependencias entre sistemas, facilitando la evolución tecnológica y soportando altos volúmenes de transacciones financieras.
Consistencia y simplicidad	El diseño debe seguir convenciones homogéneas de nomenclatura, estructuras de datos y respuestas de error, facilitando la experiencia de los consumidores internos y externos.
Observabilidad y monitoreo	Cada API debe permitir trazabilidad de extremo a extremo, con correlation IDs, métricas de rendimiento y visibilidad en tiempo real para garantizar disponibilidad y auditoría.
Cumplimiento regulatorio	Las APIs deben ajustarse a las normativas financieras vigentes, incluyendo protección de datos personales, prevención de fraudes y estándares de seguridad de la industria .
Documentación y descubrimiento	Todas las APIs deben ser visibles en el catálogo corporativo.

5. Especificaciones técnicas

5.1. Protocolos y estándares

1) Protocolo HTTP

Versión 1.1 recomendada

- **HTTP/1.1:** Mínimo recomendado para compatibilidad
- **HTTP/2:** Recomendado para mejor performance (multiplexing, compresión)
- **HTTP/3:** Opcional, para casos de alto rendimiento

2) Especificación estándar para documentar APIs REST:

OpenAPI

- Descripción de endpoints
- Parámetros, headers, body
- Respuestas y códigos de estado
- Esquemas (schemas)
- Seguridad (security schemes)

3) Uso obligatorio de HTTPS

Requisito: Todas las APIs deben exponerse exclusivamente sobre HTTPS/TLS (no se permiten endpoints HTTP).

Versiones TLS: Requerir TLS $\geq$ 1.2; preferir TLS 1.3 cuando sea posible.

Cifrado y suites: Imponer suites de cifrado fuertes (AEAD, ECDHE) y deshabilitar algoritmos obsoletos (RC4, MD5, SHA-1).

HSTS: Habilitar HSTS para servicios públicos cuando corresponda.

Renovación de certificados: Gestionar certificados via ACME/automation (ejemplos: AWS ACM, Azure Key Vault) y alertas de expiración.

4) Formato por defecto: JSON (y variantes)

Formato por defecto: application/json para APIs REST.

Soporte opcional: XML solo para compatibilidad legacy; Avro/ProtoBuf para servicios de alto rendimiento o mensajería (cuando ambos extremos lo soporten).

Content-Type: Todas las respuestas y peticiones deben especificar Content-Type y Accept.

Serialización: Definir normas sobre fechas (ISO 8601), números (uso de string para decimales precisos si aplica), y manejo de nulls.

Schema-first: Cuando se use protobuf/Avro, versionar los schemas y mantener backwards compatibility.

5) Soporte de versionado semántico

Las API siguen las reglas de **Semantic Versioning (SemVer)** para identificar y comunicar cambios en sus versiones.

Estructura: MAJOR.MINOR.PATCH

MAJOR (ej. v2.0.0)

Cambios incompatibles con versiones anteriores (breaking changes).

Ejemplo: cambiar el formato de una respuesta, eliminar un endpoint.

MINOR (ej. v1.3.0)

Cambios compatibles hacia adelante, se agregan nuevas funcionalidades.

Ejemplo: añadir un nuevo endpoint o un campo opcional en la respuesta.

PATCH (ej. v1.3.2)

Correcciones de errores o ajustes menores que no afectan la compatibilidad.

Ejemplo: corregir un type en la documentación o un bug en la validación de un campo.

Cambios mayores: (MAJOR ejemplos: ruptura de contrato, eliminación de atributos, cambios en semántica de negocio): requieren incrementar el prefijo de versión.

Cambios menores (MINOR Y PATCH ejemplos: adición de atributos opcionales, mejoras no disruptivas): se manejan dentro de la misma versión y deben mantenerse compatibles hacia atrás.

Documentación: mantener changelog claro y política de compatibilidad.

5.2. Nomenclatura y convenciones

Convenciones de Nomenclatura y Estructura

Idioma: todos los recursos, atributos y contratos deben definirse en inglés para asegurar consistencia global. Recursos en plural: /customers, /payments. Uniformidad: No mezclar idiomas. Consistencia en pluralización, evitando formas mixtas.

Manejo de Acciones en Recursos

Se debe evitar el uso de verbos en las rutas de recursos. Los verbos HTTP (GET, POST, PATCH, DELETE) son responsables de la semántica de la operación. Sin embargo, existen excepciones claras para operaciones que no corresponden al ciclo de vida estándar CRUD (Create, Read, Update, Delete).

Para acciones no-CRUD, se admiten dos patrones:

1. **Sub-recursos de acción con POST** (cuando la acción es discreta y atómica)
 - Estructura: POST /resources/{id}/action-name
 - Ejemplo: POST /payments/{id}/rollback
 - Casos de uso: revertir transacción, enviar notificación, ejecutar validación, procesar solicitud
 - Ventaja: Semántica clara y RESTful, sigue el estándar de recursos
2. **PATCH declarativo** (cuando la acción requiere contexto o cambios complejos)
 - Estructura: PATCH /resources/{id} con body { "action": "ACTION_NAME" }
 - Ejemplo: PATCH /payments/{id} con { "action": "ROLLBACK", "reason": "customer_request" }
 - Casos de uso: transiciones de estado complejas, acciones que requieren datos adicionales, cambios parciales con contexto
 - Ventaja: Permite enviar información complementaria, manejo de casos complejos

Criterios de selección entre patrones:

- Usa sub-recursos (POST) si: la acción es simple, no requiere datos adicionales, es idempotente o sin efectos secundarios.
- Usa PATCH declarativo si: necesitas enviar contexto/parámetros, la acción tiene lógica compleja, requiere auditoría detallada o cambios condicionales.

Prohibido: Usar verbos directamente en rutas como POST /payments/rollback, GET /users/export, o POST /accounts/activate.

Evitar camelCase en URLs.

Elemento	Recomendación	Ejemplo
Body (JSON)	camelCase	"firstName": "Juan"
Headers	Pascal-Case con -	Correlation-Id
Query Params	camelCase	?includeOrders=true
Path Params / URLs	kebab-case	/customer-orders/{order-id}

Estructura de Recursos

- Colección o recurso: GET /clients, GET /accounts
- Subrecurso: GET /clients/{clientId}/accounts

Nomenclatura de Campos y Objetos

Para el nombramiento de campos y objetos dentro de una especificación de API se debe utilizar obligatoriamente el Modelo de Referencia BIAN (Banking Industry Architecture Network) versión 13, con el fin de garantizar la estandarización y consistencia en el ámbito financiero. (Ver referencia: BIAN Service Domain Landscape)

En determinados casos se podrá adaptar o "tropicalizar" las denominaciones definidas por BIAN, con el propósito de mejorar la claridad semántica y mantener un orden coherente en los atributos relacionados con las APIs. Cualquier adaptación debe ser documentada explícitamente en el contrato de la API.

Ejemplo:

Campo OpenAPI	Nombre	Descripción/Formato	Ejemplo
Información general			
openapi	Openapi	La versión de Openapi es la más soportada por la mayoría de los productos 3.0.x	3.0.1
Info.Title	Nombre de la API	Caso canónico: <capa>-<service-domain>- <*personalización>-<application> Caso compuesto <capa>-en-<service-domain>- <*personalización>-<application> Notas: <personalización> es opcional y definida por el chapter.	Caso canónico sys-card-billing-and-payments-pro Caso compuesto sys-en-card-billing-and-payments-pro

Campo OpenAPI	Nombre	Descripción/Formato	Ejemplo
		<i><application> corresponde al código oficial de la aplicación.</i>	
Info.description	Descripción de la API	Describe de forma clara y funcional el propósito de la API.	API para gestionar notificaciones
Info.version	Versión	Semantic versioning x.x.x Nota: Para más detalle sobre la versión de la API, consulta la sección Versionado y ciclo de vida .	1.0.0
Información de contacto			
Info.contact.name	Equipo owner	Nombre equipo owner.	Equipo de medios de pago
Info.contact.email	Correo de soporte	Correo oficial de soporte del equipo.	mediosdepago@intercorp.com.pe
Documentación externa			
externalDocs.description	Descripción Docs	Describe el contenido de la URL de documentación.	Portal / Repositorio de APIs
externalDocs.url	URL Docs	URL del portal o repositorio de documentación.	https://portal.apis.interbank.per
Servidores (Servers)			
servers.URL	URL Base	<a href="https://<dns-gateway>/<prefix-direction>/<api-name>/<version>">https://<dns-gateway>/<prefix-direction>/<api-name>/<version>	https://apib2b.interbank.per/ext-in/ sys-card-billing-ccd/v1
servers.description	Server Description	Descripción del tipo de api gateway	API gateway con terceros
Tags de agrupación de API			
tags-name	Nombre del Tag	<Consulta/transacciones> – <entidad/dominio/funcionalidad> – <personalización>	Consulta – Payments – CCD
tags-description	Descripción del Tag	Descripción funcional del dominio agrupado	Medios de pago, Payments.

Campo OpenAPI	Nombre	Descripción/Formato	Ejemplo
operation.tag	Tags operation	Referencia al tag agrupador definido en la API	[Consultas]
paths	Path	/<behavior-qualifier>/<action> Nota: Para más detalles sobre el path, consulta la sección 5.2 Nomenclatura y convenciones donde se explican las reglas para URL y path.	/debt-simulation/execute
Method	Método	PUT: Actualización total GET: Consulta de Información POST: Creación o ejecución de operaciones. PATCH: Modificación parcial DELETE: Eliminación Nota: Para más detalles sobre los verbos, consulta la sección de Verbos HTTP estándar y semántica .	POST /debt-simulation/execute
-	Prefijo para APIs de Terceros	Prefijo: Uso /ext-in/: Consumo desde un tercero hacia la aplicación interna. /ext-out/: Consumo desde aplicación interna hacia un tercero. Ext = external, extranet. APIs Internas No se utiliza prefijo de dirección.	/ext-in/ /ext-out/
Resumen y descripción Operaciones			
summary	Resumen operación	Resumen breve de la operación.	Simulación de crédito
description	Descripción operación	Descripción funcional detallada.	Realiza la simulación de un crédito hipotecario.
URI	URI	<method> /<prefix-direction>/<api-name>/<version>/<path> Nota:	POST /ext-in/sys-card-billing-andpayments-ccd/v1/debt-simulation/execute

Campo OpenAPI	Nombre	Descripción/Formato	Ejemplo
		<i>Para más detalle sobre la versión de la API, consulta la sección Versionado y ciclo de vida.</i> <i>Para mas detalles sobre el path consulta la sección 5.2 Nomenclatura y convenciones.</i>	
-	Endpoint	https://<dnsgateways>/<URI>	https://apib2b.interbank.per/ext-in/ sys-card-billing-ccd/v1/debt-simulation/execute

Guía para completar la información del OpenAPI.

<Campo>	Referencia
<capa>	Ver en documento tabla Tipo de API
<service-domain>	Ver en anexo BIAN Service domain Landscape
<business-domain>	Ver en anexo BIAN Service domain Landscape
<behavior-qualifier>/<action>	Ver en anexo BIAN Service domain Landscape
<aplication>	Ver en referencias Catalogo de aplicaciones.
<Personalización>	La personalización la da el chapter.
<cod-organización>	Ver en documento tabla Organización
<dns-gateway>	Indicar siempre el nombre de dominio del gateway en lugar de la dirección IP. Ver arquitectura de solución.

```
openapi: 3.0.1
info:
  title: sys-card-billing-and-payments-ccd
  description: API para simulación, consulta y detalle de créditos. Compliance con API style guide.
  version: 1.0.0
servers:
  - url: https://api.interbank.pe/v1/ccd
    description: API Gateway Production
  - url: https://api.uat.interbank.pe/v1/ccd
    description: API Gateway Production
tags:
  - name: Credit
    description: Operaciones relacionadas con la gestión de pagos.
paths:
  /credits/simulations:
    post:
      Scan | Try it | Audit
      summary: Simular crédito
      description: Realiza una simulación de crédito según criterios enviados.
      operationId: simulateCredit
      tags: [Credit]
      parameters:
        - $ref: '#/components/parameters/Consumer-Id'
        - $ref: '#/components/parameters/Device-Id'
        - $ref: '#/components/parameters/Ip-Origin'
```

5.3. Modelado de recursos

Regla: Plantear recursos como entidades de dominio independientes; usar sub-recursos para relaciones o colecciones asociadas.

Ejemplo:

Recursos: /clients, /accounts

Sub-recurso: /customers/{id}/accounts → cuentas de un cliente.

Uso de identificadores únicos IDs: Preferir IDs UUID v4 o identificadores que no expongan información sensible.

Estabilidad: Los IDs no deben ser cambiados; no exponer IDs secuenciales fácil de enumerar (evitar riesgos de scraping/enumeration).

Atributos obligatorios y opcionales:

Se debe basar en el diccionario de datos corporativo, gobernado conjuntamente por equipos de Gobierno de Datos y Delivery.

Definición clara: Cada recurso en la especificación debe marcar campos required y optional.

Default & validation: Especificar valores por defecto y reglas de validación (longitud, formato, rangos, regex para identificadores).

Ejemplo OpenAPI fragmento

```
components:
  schemas:
    Cliente:
      type: object
      required:
        - clientId
        - name
      properties:
        clientId:
          type: string
          format: uuid
        name:
          type: string
        birthDate:
          type: string
          format: date
```

Consideraciones financieras: Campos monetarios: usar minor units (p. ej. centavos) o strings con precisión decimal; documentar la moneda (ISO 4217) y reglas de redondeo.

5.4. Operaciones

Verbos HTTP estándar y semántica

Método	Descripción	Acción
GET	Obtener información de una entidad, idempotente. Los parámetros deben ir como Query String.	Retrieve Notify
POST	Lanzan la generación de una nueva entidad. No idempotente. Los parámetros de input van en el cuerpo del mensaje. Manejar Idempotency-Key.	Create Initiate Register Evaluate Provide Feedback Exchange Capture Execute Grant
PUT/PATCH	Actúan sobre una entidad modificación parcial o total.	Update Configure Activate Control
DELETE	Eliminación de una entidad	Delete

Datos sensibles en la URI: Toda información PCI/Datos sensibles debe viajar únicamente en el cuerpo encriptado o cabeceras seguras (ejemplos: PAN de tarjetas, números de cuentas). Si una operación de consulta GET tiene como parámetro de entrada un dato sensible, se convierte automáticamente en POST.

Uso de Path Parameters para identificar recursos en operaciones GET y DELETE:

En las operaciones que actúan sobre un recurso individual (por ejemplo, obtener o eliminar un registro específico), el identificador único del recurso debe formar parte de la ruta (URI Path) como un Path Parameter, no como un Query Parameter.

Los recursos deben tener una URI única y jerárquica que los identifique. El uso de PathParam mejora la semántica REST, la legibilidad, la trazabilidad en logs y la conformidad con los principios de diseño RESTful definidos en RFC 9110 y OpenAPI Specification.

✓ GET /recurso/{id} → Recupera un recurso específico.

✓ DELETE /recurso/{id} → Elimina un recurso específico.

✗ GET /recurso?id=123

✗ DELETE /recurso?id=123

Idempotencia y diseño transaccional

Idempotency-Key: Para operaciones financieras (pagos, transferencia), requerir Idempotency-Key en headers para evitar duplicados (llave única evitar key repetida). Documentar TTL (Time to Live) de la clave.

Transacciones distribuidas: Evitar transacciones ACID cruzando microservicios; preferir sagas o patterns compensatorios.

Paginación, filtros y ordenamiento

Paginación: preferir cursor-based pagination para grandes volúmenes; limit/offset aceptado como fallback.

Cursor example: GET /transactions?cursor=eyJhxrtd&limit=100

Filtros: Usar parámetros claros y estandarizados (?state=pending&from=2025-01-01) y documentar combinatoria y límites.

Ordenamiento: sort=fecha:desc,importe:asc.

No programar lógica de negocio en la capa de exposición de APIs.

El API Gateway no debe componer, transformar ni agregar lógica compleja.

La exposición debe ser delgada y transparente, delegando la lógica a microservicios o sistemas backend.

6. Estructura de mensaje.

Los campos necesarios para procesar el mensaje deben estar en el body del mensaje y/o como parámetros en el query string dependiendo del método.

Los campos de los mensajes del request, response y queryString deben manejarse en formato camelCase y en inglés.

- Revisar el diccionario de datos (*Ver referencia:* Diccionario de Datos)

- Debe indicar claramente el uso.

- Se debe utilizar **ISO20022**.

- Si no se encuentra la definición en el ISO se utilizará la traducción directa del campo.

Los campos minimos obligatorios que toda API debe enviar y recibir son los siguientes:

Estructura del mensaje request

Headers	Tipo	Descripción	Ejemplo
Channel-Id	Inicio	Canal donde se originó la transacción (Web, MFE, Movil, Agencia) + Código de aplicación.	Movil_SMP
User-Id	Inicio	Usuario de la persona (colaborador o cliente) logueada en la aplicación donde inicia la transacción.	s34426
Device-Id	Inicio	Identificador del dispositivo donde inicio la transacción. Móviles: UUID Nativo (IDFV iOS, AndroidID) - 36 caracteres máx. Web/SPA: Fingerprint (Hash de atributos) - 36 caracteres máx.	CF3C0766-2313-4CA2-8805-72F073539160
Idempotency-Key	Inicio	Llave para identificar la transacción y evitar que el backend pueda registrar transacciones duplicadas. UUID v4 36 caracteres máx.	885e3a07-8e6f-4c5b-8f1d-9e6a0d4b3c2a
User-Agent	Inicio	SO/Navegador del cliente	Mozilla/5.0 (iPhone; CPU iPhone OS 16_6 like Mac OS X) AppleWebKit/605.1.15 (KHTML, like Gecko) Mobile/15E148
Accept	Variable	Formatos que el cliente acepta	Accept: application/json
Accept-Encoding	Variable	Compresión soportada	Accept-Encoding: gzip, deflate
Content-Type	Variable	Tipo de contenido enviado en body con metodos POST/PUT/PATCH	Content-Type: application/json
Content-Length	Variable	Tamaño del cuerpo. Cuando hay body.	Content-Length: 1234
Feature-Id	Inicio	Identificador funcional de la feature o flujo de negocio.	wishlist
Feature-Step-Id	Inicio	Paso específico dentro de la feature o flujo.	add-to-cart
Timestamp	Variable	Fecha y hora en formato YYYY-MM-DDThh:mm:ss.sssZ ISO ISO 8601 del inicio de la última invocación (Auditoría).	2025-12-08T22:55:00.123Z
**Request-Timestamp	Inicio	Fecha y hora en formato YYYY-MM-DDThh:mm:ss.sssZ ISO ISO 8601 del inicio de la transacción (Auditoría).	2025-12-08T22:55:00.123Z
Traceparent	Variable	W3C. Trace-Id persiste (Fijo), Span-Id cambia (Variable). <version>-<traceld>-<spanID/parentId>-<flag_rastreo>	00-4bf92f3577b34da6a3ce929d0e0e4736-0000000000000001-01

*X-Forwarded-For	Variable	Lista acumulativa de IPs. Se envía automáticamente cuando se pasa por proxy, CDN, el cliente no lo ingresa.	201.234.11.45, 10.0.1.5
*CF-IP-Source	Inicio	Ip original del cliente. Se envía automáticamente cuando se pasa por cloudfare, el cliente no lo ingresa.	201.234.11.45
Ip-Origin	Inicio	Ip de componente que inicia la transacción. Debe ser seteado por la aplicación que inicia la transacción.	201.234.11.45
Device-Type	Inicio	Tipo de dispositivo (tablet, smartphone, desktop, wereable, iot-device)	smartphone
Consumer-Id	Variable	Identificador de la aplicación consumidora.	ASSI
**Session-Id	Inicio	Identificador de sesión generado por el frontend del canal consumidor.	a3eebc33-2c0b-1ef3-bb6d-7bb9bd780a34
Message-Id	Variable	Identificador del mensaje generado en el canal consumidor. Request-ID único (UUID) por salto. 36 caracteres máx.	a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11
Authorization	Variable	Bearer <Token JWT>	AAIgYzhjNWI1YTc1NDY0MzdHO TQxYTczODdkOTgzNW Q5NGHRV-ohDrT8VElf1z
Subscription Key	Variable	Subscription key	27535b14116f311db1a7546c034b6416
Body {estructura propia de API}	Variable	camelCase, Ingles e ISO 20022. Cada campo del body debe definir explícitamente obligatoriedad , tipo de dato, así como la longitud mínima y máxima permitida, de acuerdo con las reglas del contrato y las restricciones de negocio aplicables.	schema: type:object body: nombre: type: string minLength: 1 maxLength: 100 required: - nombre

*Headers que se autocompletan automáticamente, no son obligatorias depende de por que capa pasan.

** Obligatorio en APIs de experiencia.

Inicio: El valor que va en esos campos vienen desde la aplicación que inicio la transacción.

Variable: El valor se actualiza en cada salto de la transacción al consumir las apis.

Estructura del mensaje de response

Headers	Campo	Descripción
	Message-Id	Es el mismo messageId que envió la aplicación consumidora
	Traceparent	Contiene información del traceId y el parentId en el formato: <version>-<traceId>-<parentId>-<flag_rastreo> Ejemplo: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01
	Content-Type	Tipo de contenido enviado en body.
	Response-Timestamp	Response-Timestamp (ISO 8601)

Campo	Descripción
Status Line	Aquí vienen los códigos de respuesta del estándar de HTTP como el 200, 209 , 404, 500, etc es la primera línea que debe leer la aplicación para entender el mensaje. Para mas detalle ver la sección 7.1. Códigos de respuesta HTTP

```
responses:
  '201':
    description: Usuario creado exitosamente
    headers:
      Message-Id:
        $ref: '#/components/headers/MessageId'
      Traceparent:
        $ref: '#/components/headers/Traceparent'
      Content-Type:
        $ref: '#/components/headers/ContentType'
      Response-Timestamp:
        $ref: '#/components/headers/ResponseTimestamp'
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/UserResponse'
        example:
          id: 550e8400-e29b-41d4-a716-446655440000
          name: Juan Pérez
          email: juan@ejemplo.com
          age: 30
          createdAt: '2026-02-11T10:30:00.451Z'
```

Nota: Los campos especificados son los mínimos necesarios, es posible agregar campos personalizados en los headers de request y de response según la necesidad.

Estructura del cuerpo del mensaje.

El cuerpo de la solicitud y de la respuesta (Body) de todas las APIs debe adherirse a las siguientes directrices:

- **Caso de Nomenclatura:** Se utilizará la convención camelCase para todos los nombres de campos (atributos) dentro de la estructura de datos.
- **Idioma:** Se utilizará inglés para la denominación de todos los campos.
- **Marco de Referencia:** La estructuración del diccionario de datos debe seguir, en la medida de lo posible, las directrices del estándar **ISO 20022**, especialmente en la definición de tipos de datos financieros y transaccionales.

Excepción para APIs de Sistema de Terceros

Se establece una excepción a la regla de estructuración estándar para los casos específicos de **APIs de Sistema** cuya función primaria sea interactuar con servicios implementados **fuera de los límites de la organización** (sistemas *legacy* o de terceros).

- **Criterio de Excepción:** El cuerpo del mensaje de estas APIs de Sistema puede mantener la estructura y nomenclatura del proveedor externo.
- **Justificación:** El objetivo de esta excepción es **minimizar la sobrecarga de desarrollo** y **reducir la latencia** al evitar transformaciones de datos innecesarias entre la API de Sistema y el *endpoint* de destino.

7. Manejo de errores

7.1. Códigos de respuesta HTTP

Uso correcto de códigos HTTP Status code:

Estándar

Código	Categoría	Descripción
2xx	Códigos de Éxito	La solicitud fue procesada correctamente por el servidor. 200 OK 201 Created 204 No Content. 209 (Error de negocio)
3xx	Redirección	La solicitud requiere una acción adicional por parte del cliente. 301 Moved Permanently 304 Not Modified.
4xx	Errores del consumidor	La solicitud presenta errores de entrada, autenticación o reglas funcionales del cliente. 400 Bad Request 401 Unauthorized 403 Forbidden 404 Not Found 408 Request timeout (el cliente no complete la solicitud a tiempo) 409 Conflict (Error de Negocio) 415 El servidor no soporta el Content-Type de la solicitud 422 Unprocessable Entity (Formatos, validaciones) 429 Too many request.
5xx	Errores del servidor	El servidor no pudo procesar la solicitud debido a errores internos o dependencias externas. 500 Internal Server Error 502 Bad Gateway 503 Service Unavailable 504 Gateway Timeout.

Personalizados Interbank:

Para los mensajes personalizados por cada aplicación se usará el campo code con la siguiente estructura:

<DOMINIO>-<codigoHTTP>-<NRO-Correlativo>

Dominio	Descripción
BUS	Business (Negocio) validaciones funcionales reglas de negocio.
VAL	Validation (Validación) Errores de formato o entrada de datos
SEC	Security (Seguridad) Errores de autenticación, autorización o permisos.
TRX	Transaction (Transacción) Errores en procesos transaccionales o de operación.
SYS	System (Sistema/Técnico) Errores técnicos, excepciones en backend.
INT	Integration (Integración) Fallas entre componentes o middleware, transformaciones, colas.
EXP	Experience. Codigos de error que se manejan en aplicaciones front-End.

CódigoHTTP	Descripción	Ejemplo
409	Código estándar http error semántico	La cuenta no existe.
209	Código de error de negocio personalizado en IFX	El cliente no cuenta con saldo disponible.

Se pueden definir **códigos personalizados para los errores de negocio**, con el fin de **mejorar el control, trazabilidad y gestión de incidencias** en las APIs.

7.2. Estructura de cuerpo del mensaje de error

La estructura del mensaje de respuesta de error debe seguir el estándar de Problem Details.

Problem Details es un formato estándar de error HTTP definido en la **RFC 7807**, que busca unificar la estructura de las respuestas de error en APIs REST.

Estructura base:

Body	Descripción
<pre>{ "type": "https://banco.com/errors/invalid-account", "title": "Cuenta inválida", (Obligatorio*) "status": 400, (Obligatorio) "detail": "El número de cuenta no existe ", (Obligatorio) "instance": "/transferencias", "code": " BUS-422-01", (Obligatorio) "correlationId": "af12c9e7-33dd-4a9a-aabc-9f2ed4cda843" (Obligatorio*) "timestamp": "2025-10-20T15:22:45Z" (Obligatorio*) }</pre>	Estructura estándar del mensaje de error en el body basado en Problem Details de OpenAPI.

(*)Campos NO obligatorios para APIs de experiencia canales/BFF.

type Identifica el tipo de error. Puede apuntar a documentación o ser un URI simbólico.

title Breve descripción legible del error.

status Código HTTP asociado al error.

detail Descripción más detallada del error.

Instance Ruta del recurso donde ocurrió el error.

code: código único para facilitar soporte y mapeo a documentos internos.

correlacionId: imprescindible para trazabilidad entre logs y soporte.

Timestamp: Fecha y hora del momento que sucedió el error.

No exponer: stack traces, SQL errors, o detalles que puedan ayudar a un atacante.

Se puede agregar información adicional en el body sin exponer información sensible que ayude al atacante.

```
responses:
  '400':
    description: Solicitud inválida - Error en formato o datos requeridos
    headers:
      Message-Id:
        $ref: '#/components/headers/MessageId'
      Traceparent:
        $ref: '#/components/headers/Traceparent'
      Content-Type:
        $ref: '#/components/headers/ContentType'
      Response-Timestamp:
        $ref: '#/components/headers/ResponseTimestamp'
    content:
      application/problem+json:
        schema:
          $ref: '#/components/schemas/ProblemDetails'
        example:
          type: https://api.ejemplo.com/problems/invalid-request
          title: Invalid Request
          status: 400
          detail: El campo 'email' es requerido
          instance: /usuarios
          traceId: abc123-def456
```

Buenas prácticas

- Documentar posibles códigos de error por endpoint en OpenAPI.
- Ofrecer enlaces a documentación para errores recurrentes.

8. Seguridad

8.1. Autenticación y autorización

El cumplimiento de los siguientes protocolos de seguridad debe ser aplicado a todas las APIs, según el escenario de consumo.

Flujo de Seguridad	Escenario de Uso Obligatorio	Propósito
OAuth 2.0 Authorization Code Flow	Delegación de Autorización (Usuario a Máquina)	Debe utilizarse cuando una aplicación cliente (web, móvil, SPA) necesita acceder a recursos de API en nombre de un usuario final autenticado.

Flujo de Seguridad	Escenario de Uso Obligatorio	Propósito
OAuth 2.0 Client Credentials Flow	Comunicación Máquina a Máquina (M2M)	Debe utilizarse cuando un servicio o microservicio necesita acceder a recursos de otra API sin la intervención de un usuario.
mTLS (Mutual Transport Layer Security)	Servicios Críticos o de Backend	Recomendado para asegurar las comunicaciones entre servicios back-end críticos, ofreciendo autenticación bidireccional al nivel de transporte.

Esquema de seguridad

Flujo	Esquema OpenAPI
Esquema OAuth2 Client Credential <i>Flujo para acceso de servicio (M2M)</i>	components: securitySchemes: OAuth2ClientCredentials: type: oauth2 description: Requerido para la comunicación segura M2M flows: clientCredentials: # URL para la obtención del Access Token. tokenUrl: https://auth.ejemplo.com/oauth/token scopes: read: Permite la ejecución de funciones de solo lectura. write: Permite la modificación de datos
Esquema OAuth2 Authorization Code <i>Flujo para acceso de usuario SPA/Movil (Delegación)</i>	components: securitySchemes: OAuth2AuthorizationCode: type: oauth2 description: Requerido para acceder a recursos en nombre de un usuario final. flows: authorizationCode: authorizationUrl: https://auth.ejemplo.com/oauth/authorize Token tokenUrl: https://auth.ejemplo.com/oauth/token scopes: read: Permite la lectura de datos del perfil del usuario. write: Permite la modificación de recursos del usuario.

- **Scopes & claims:** Definir scopes finos y claims mínimos necesarios (principio de menor privilegio).

Controles Obligatorios de autorización para evitar BOLA y BFLA.

- **BOLA** (Broken Object Level Authorization) Valida permisos en cada solicitud verificando que el usuario autenticado es el propietario o tiene roles autorizados para acceder al recurso específico. No confiar en IDs provistos por el cliente sin verificación de ownership.
- **BFLA** (Broken Function Level Authorization): Validar roles/scopes por cada operación.
- **Token management:** preferir JWT pero validar revocación (token introspection) para accesos críticos.

8.2. Gestión segura de tokens

- **Expiración:** access tokens cortos (ej.: 5-15 min), refresh tokens con restricciones y revocación.
- **Storage:** no almacenar tokens en lugares inseguros; para SPAs usar Authorization Code Flow + PKCE.

- **Rotación:** mecanismos de rotate keys/secrets automáticos; rotación planificada y automatizada.

8.3. Encriptación en tránsito y en reposo

- **Transito:** TLS obligatorio; certificados gestionados centralmente.
- **Reposo:** cifrado de datos sensibles en bases y backups (KMS gestionado).
- **Field-level encryption:** para datos extremadamente sensibles (números de tarjeta, Lineamiento PCI).

8.4. Exposición de Datos

- **No devuelvas más datos de los necesarios:** Filtra campos sensibles en todas las respuestas. Valida permisos a nivel de campo.
- **Mensajes genéricos:** Los mensajes de error deben ser genéricos, no se debe devolver códigos internos de dominio. Esto aplica para APIs de experiencia.

8.5. Protección contra abuso

- **Rate limiting y throttling** por consumidor y por API; políticas diferenciadas para terceros o partners.
- **Seguridad en la cadena:** Incluir validaciones de seguridad en los pipelines de DevSecOps CI/CD.
- **Scan** código estático y pruebas ethical hacking periódicas.

8.6. Otros controles recomendados

Políticas: CORS restrictivas (Web/MFE) y protección CSRF si aplica.

Configuración de la Cache: Cache-Control: no-store para respuestas con datos sensibles.

9. Versionado y ciclo de vida

9.1. Estrategia de versionado

Todas las API deben seguir Semantic Versioning (SemVer) para identificar, comunicar y gestionar los cambios de manera clara y predecible a lo largo del tiempo.

El esquema de versionado sigue el formato:

MAJOR.MINOR.PATCH

9.2. Semántica de versiones

MAJOR (ej. v2.0.0)

Representa los cambios incompatibles con versiones anteriores (breaking changes) que requieren intervención por parte de los consumidores.

Ejemplos:

- Cambio en la estructura o formato de un response.
- Eliminación o renombramiento de endpoints existentes.
- Eliminación o cambio de tipo de atributos obligatorios.
- Cambios en la semántica del negocio que alteran el comportamiento esperado.

MINOR (ej. v1.3.0)

Representa mejoras compatibles hacia atrás, donde se agregan nuevas capacidades sin afectar a los consumidores existentes.

Ejemplos:

- Adición de nuevos endpoints.
- Inclusión de nuevos atributos opcionales en request y response.
- Nuevas funcionalidades que no modifican contratos existentes.

PATCH (ej. v1.3.2)

Representa correcciones o ajustes no disruptivos que no alteran el contrato externo de la API.

Ejemplos:

- Corrección de errores de validación.
- Ajustes en la documentación OpenAPI.
- Corrección de reglas internas sin impacto externo.

9.3. Clasificación de Cambios.

Cambios Mayores (MAJOR)

Incluyen rupturas de contrato, eliminación de atributos, cambios en tipos de datos o modificaciones en la semántica del negocio.

Estos cambios requieren obligatoriamente incrementar la versión MAJOR.

Cambios Menores (MINOR y PATCH)

Incluyen extensiones compatibles y mejoras no disruptivas.

Deben mantenerse totalmente compatibles hacia atrás dentro de la misma versión MAJOR.

9.4. Estrategia de Versionado

- Regla general (por defecto): La versión MAJOR debe exponerse explícitamente en la URL Base de la API para identificar la generación del contrato consumido.

Ejemplos:

/api/v1/customers

/api/v2/customers

- La versión MAJOR solo debe cambiar cuando se introducen *breaking changes*.

9.5. Comunicación de Versiones MINOR y PATCH

- Las versiones MINOR y PATCH no deben incluirse en la URL.
- Estos cambios deben comunicarse mediante:

El campo info.version del contrato OpenAPI / Swagger

Documentación publicada en el CodeIBK, Developer Portal y el changelog correspondiente.

9.6. Garantía de compatibilidad

Mientras los consumidores permanezcan en la misma línea MAJOR (por ejemplo, v1.x.x) se garantiza que:

- No se introducirán rupturas de contrato.
- Las nuevas capacidades serán opcionales.
- Las correcciones serán transparentes y no disruptivas.

9.7. Deprecación y retiro

- **Proceso formal:** Anunciar con antelación 90 días según impacto.
- **Deprecado** El estado de la API cambia cuando se despliega en Producción la nueva versión.
- **Notificación:** Se debe notificar a los consumidores de la versión deprecada el deadline para que actualicen.
- **Cuarentena:** Finalizado el deadline para poder usar la versión deprecada, esta se inactiva por un periodo de tiempo antes de que sea eliminada.
- **Retiro:** Terminada la cuarentena se procede a eliminar y se actualiza el estado en el catálogo.

10. Calidad y pruebas

10.1. Pruebas

- **Unitarias:** Pruebas unitarias para validar lógica del servicio.
- **Integración:** Pruebas end-to-end en entornos similares a producción.

10.2. Pruebas de carga y resiliencia

- **Load testing:** definir SLO/SLA con thresholds y escenarios.
- **Chaos/Resilience:** circuit breakers, retries con backoff, timeouts; pruebas que simulen fallas.

11. Observabilidad

11.1. Logs estructurados

- **Formato:** JSON log con campos estándar no sensibles (timestamp, level, service, operation, correlationId, etc). Enviar a New Relic.
- **Centralización:** Enviar a QRadar información de accesos y permisos.
- **Retención:** política de retención según regulación.

11.2. Trazabilidad distribuida

- **Correlation IDs:** obligatorio en todas las requests; propagar Correlation-Id, Message-Id y context tracing (W3C TraceContext).
- **Tracing:** instrumentar con OpenTelemetry, capturar spans relevantes.

11.3. Métricas

- Latencia P50/P95/P99, throughput, error rate, saturation/queue length.
- Exportar métricas a NewRelic y configurar alertas.

12. Gobierno y cumplimiento

12.1. Revisión por Chapter

- **Conformidad:** todas las APIs nuevas deben pasar una revisión a nivel de diseño/implementación/seguridad/regulatorio.
- **Checklist:** Seguridad, logging, SLA, tests, políticas, permisos.

12.2. Checklist de cumplimiento

- Seguridad (authz/authn), Documentación (OpenAPI), Estilo (naming/response), Tests (unit, contract, load), Observability (tracing, logs).

12.3. Roles y responsabilidades

- **Propietario del API:** Responsable de dar soporte a la API y el primer punto de contacto ante cualquier incidente.
- **Equipo de Gobierno y Delivery:** Equipo dueño del catálogo empresarial de APIs, encargado de revisiones y cumplimiento.
- **Equipo de soporte de API:** Equipo encargado de administrar la plataforma de APIs.
- **Chapter:** Chapter del equipo de Gobierno y Delivery encargado de validaciones y conformidades asignado a cada crew.
- **Arquitecto de Software:** Para las APIs de experiencia y de aplicación es el responsable de validar y dar conformidad al diseño y a las excepciones.

13. Ejemplos y antipatrones

13.1. Buenas prácticas (ejemplos)

- **Idempotencia:** pagos con Idempotency-Key.
- **Error handling:** devolver 409 Conflict con payload explicativo cuando conflicto de deduplicado.
- **Documentación:** OpenAPI con ejemplos concretos de request/response.

13.2. Antipatrones a evitar

- **Single mega-endpoint** que hace todo (/api?action=...) — dificulta evolución y gobierno.
- **Exponer esquema de BD** directamente en la API.

- **Tamaño de respuesta sin paginar** en endpoints con potencial de grandes volúmenes.
- **Errores con stack traces** visibles al cliente.

14. Glosario y referencias

14.1. Definiciones clave

- **Recurso:** entidad representada en la API (cliente, cuenta, transacción).
- **Contrato:** especificación formal del API (OpenAPI/AsyncAPI).
- **Endpoint:** URL que expone una operación.
- **Payload:** cuerpo de la request/response.
- **Token:** credencial temporal para autorización.

14.2. Referencias y estándares

- **OpenAPI Initiative.**
OpenAPI Specification, Version 3.1.0.
The Linux Foundation, 2021.
Disponible en: <https://spec.openapis.org/oas/v3.0.3>
- **IETF.**
RFC 7231: Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content.
Internet Engineering Task Force, 2014.
Disponible en: <https://datatracker.ietf.org/doc/html/rfc7231>
- **JSON Schema.**
JSON Schema Specification (Draft 2020-12).
Disponible en: <https://json-schema.org>
- **OAuth 2.0 Framework.**
RFC 6749: The OAuth 2.0 Authorization Framework.
IETF, 2012.
Disponible en: <https://datatracker.ietf.org/doc/html/rfc6749>
- **OWASP Foundation.**
OWASP API Security Top 10 – 2023.
Open Worldwide Application Security Project, 2023.
Disponible en: <https://owasp.org/API-Security>
- **W3C.**
Trace Context Recommendation.
World Wide Web Consortium, 2023.
Disponible en: <https://www.w3.org/TR/trace-context/>
- API-Led Connectivity
<https://www.mulesoft.com/es/lp/whitepaper/api/api-led-connectivity>
<https://www.ibm.com/think/topics/api-led-connectivity>

- **OpenTelemetry Project.**

OpenTelemetry Specification, Version 1.25.0.

Cloud Native Computing Foundation (CNCF), 2024.

Disponible en: <https://opentelemetry.io/docs/specs>

- **Catálogo de Aplicaciones**

<https://interbankpe.sharepoint.com/:x:/r/sites/ArquitecturaTI/Catlogos/1.%20Aplicaciones.xlsx?d=w90abb6b2e614751bdf546e0a9425398&csf=1&web=1&e=2dZbSy>

- **Lineamientos de seguridad de APIs**

<https://interbankpe.sharepoint.com/:b:/r/sites/ArquitecturaTI/Lineamientos/Integraci%C3%B3n/ARQINT%20Lineamiento%20cloud%20v1.2.pdf?csf=1&web=1&e=2eV3rQ>

- **BIAN Service domain Landscape**

<https://interbankpe.sharepoint.com/:b:/r/sites/ArquitecturaTI/Lineamientos/Integraci%C3%B3n/BIANV13.xlsm?csf=1&web=1&e=2eV3rQ>

- **Diccionario de datos**

<https://interbankpe.sharepoint.com/:b:/r/sites/ArquitecturaTI/Lineamientos/Integraci%C3%B3n/Diccionario%20de%20datos%20v3.xlsx?csf=1&web=1&e=2eV3rQ>